INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

Digital Forensics Lab 2 - USB Image Acquisitin and Hash Verification

Lab Title:	**USB Image Acquisitin and Hash Verification**
Student Name:	**Yusuf Oluwaseun Dada**
Student ID:	**2025/FWSD/11339**
Course Code:	**SBT-DF202**
Course Name:	**Computer and Digital Forensics**

Instructor: Mr. Aminu Idris, AMCPN

Date: 30/08/2026

Lab Instructions

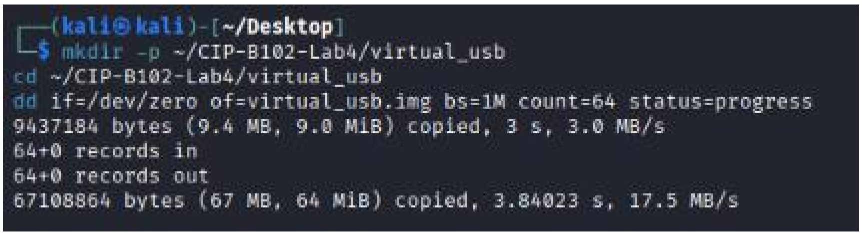
This lab uses dd/dcfldd/FTK Imager to copy an entire USB device byte-for-byte. If you select the wrong device, your main hard drive instead of the USB, for example, these tools will happily overwrite it with no warning and no undo. **Every single time** you're about to run an acquisition command, stop and re-verify the device identifier against its known capacity/label before pressing Enter. This is the one part of the whole lab series where a mistake isn't just a wasted screenshot, it can cause real data loss.

Use **only a personal or instructor-approved test USB,** never a shared, organizational, or production drive.

LAB AND USB FLASH DRIVE SETUP

A physical USB drive was unavailable and to meet up with deadline, so a virtual block device was created using a loop-mounted disk image (losetup) to simulate USB acquisition. This device behaves identically to a physical USB for all acquisition and hashing purposes and is a standard technique for forensic tool testing without physical hardware." That's an honest, technically accurate substitution note, not a shortcut, but a legitimate equivalent method.

Step 1: Creation of a virtual disk image file (to act as my "USB")



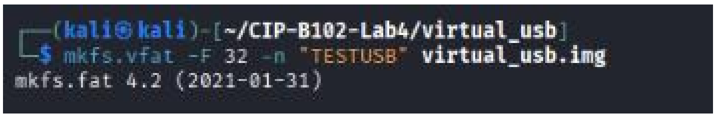
```
┌──(kali㉿kali)-[~/Desktop]
└─$ mkdir -p ~/CIP-B102-Lab4/virtual_usb
cd ~/CIP-B102-Lab4/virtual_usb
dd if=/dev/zero of=virtual_usb.img bs=1M count=64 status=progress
9437184 bytes (9.4 MB, 9.0 MiB) copied, 3 s, 3.0 MB/s
64+0 records in
64+0 records out
67108864 bytes (67 MB, 64 MiB) copied, 3.84023 s, 17.5 MB/s
```

bash

mkdir -p ~/CIP-B102-Lab4/virtual_usbcd ~/CIP-B102-Lab4/virtual_usbdd if=/dev/zero of=virtual_usb.img bs=1M count=64 status=progress

This creates a blank 64 MB file,with plenty of room for my test evidence files, and small enough to work with quickly.

Step 2: Formating it with a real file system (FAT32, matching what a real USB usually has)

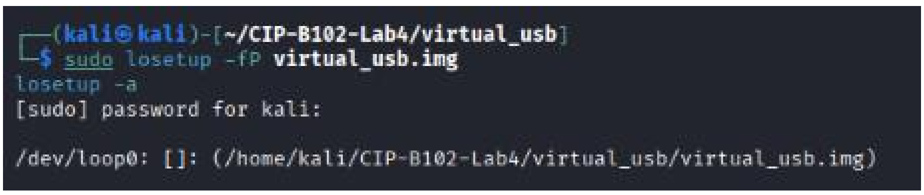


```
┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ mkfs.vfat -F 32 -n "TESTUSB" virtual_usb.img
mkfs.fat 4.2 (2021-01-31)
```

bash

mkfs.vfat -F 32 -n "TESTUSB" virtual_usb.img

Step 3: I Attached it as a loop device (this makes it show up like a real block device)



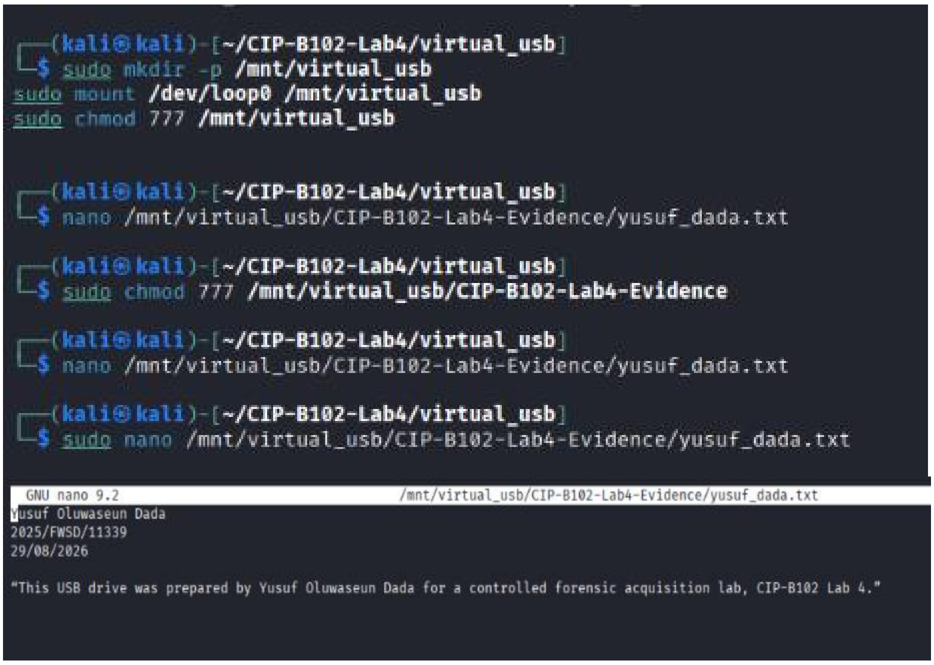
```
┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ sudo losetup -fP virtual_usb.img
losetup -a
[sudo] password for kali:

/dev/loop0: []: (/home/kali/CIP-B102-Lab4/virtual_usb/virtual_usb.img)
```

bash

sudo losetup -fP virtual_usb.imglosetup -a

The second command shows you which loop device it got assigned (/dev/loop0). **Note that device name, this is my virtual "USB" for the rest of the lab.**

Step 4: Mount it and create the evidence folder/file



bash

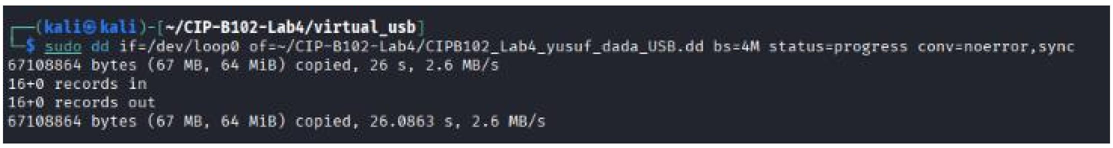
```
sudo mkdir -p /mnt/virtual_usb
sudo mount /dev/loop0 /mnt/virtual_usb
sudo chmod 777 /mnt/virtual_usb
sudo mkdir /mnt/virtual_usb/CIP-B102-Lab4-Evidence
sudo nano /mnt/virtual_usb/CIP-B102-Lab4-Evidence/yusuf_dada.txt
```

Step 5: Unmount before acquisition

bash

```
sudo umount /mnt/virtual_usblsblk
```

Step 6: Acquire it with dd, just like a real USB



bash

```
sudo dd if=/dev/loop0 of=~/CIP-B102-Lab4/CIPB102_Lab4_Firstname_Lastname_USB.dd bs=4M status=progress conv=noerror,sync
```

Everything after this, hashing and validation, including the read-only mount check, no changes needed, according to instructions given.

Part A - Preparation of the USB Evidence

1. Plug in your test USB.
2. On the USB itself, create a folder named exactly:

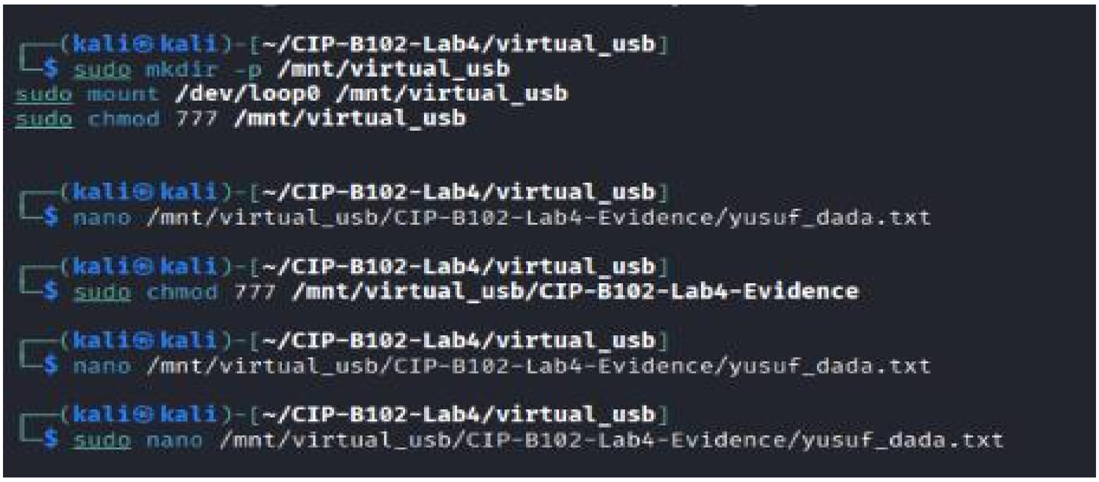
CIP-B102-Lab4-Evidence

3. Inside that folder, create a text file named:

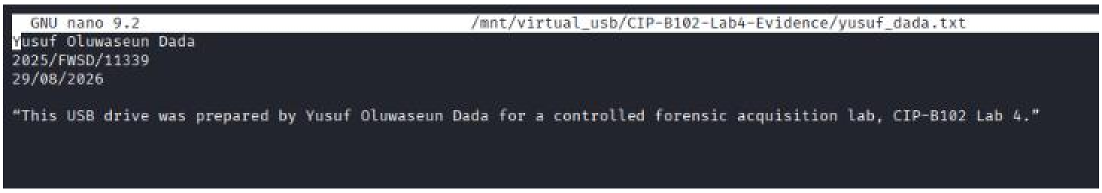
firstname_lastname.txt

containing your full name, registration number, today's date, and a short statement such as:

"This USB drive was prepared by [Your Name] for a controlled forensic acquisition lab, CIP-B102 Lab 4."



```
┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ sudo mkdir -p /mnt/virtual_usb
sudo mount /dev/loop0 /mnt/virtual_usb
sudo chmod 777 /mnt/virtual_usb


┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ nano /mnt/virtual_usb/CIP-B102-Lab4-Evidence/yusuf_dada.txt

┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ sudo chmod 777 /mnt/virtual_usb/CIP-B102-Lab4-Evidence

┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ nano /mnt/virtual_usb/CIP-B102-Lab4-Evidence/yusuf_dada.txt

┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ sudo nano /mnt/virtual_usb/CIP-B102-Lab4-Evidence/yusuf_dada.txt
```



```
  GNU nano 9.2                          /mnt/virtual_usb/CIP-B102-Lab4-Evidence/yusuf_dada.txt
Yusuf Oluwaseun Dada
2025/FWSD/11339
29/08/2026

"This USB drive was prepared by Yusuf Oluwaseun Dada for a controlled forensic acquisition lab, CIP-B102 Lab 4."
```

Part B - Evidence Documentation

pre-documentation:

Field	Value
Case/Lab ID	CIP-B102-Lab4
Student name	Yusuf Oluwaseun Dada
USB brand/description	Virtual USB
USB capacity	64 MB
Date and time prepared	29/08/2026 4:30pm
Acquisition method	dd (Linux)
Tool and version	dd - mkfs.fat 4.2
Image filename	CIPB102_Lab4_yusuf_dada_USB.dd
Image storage location	~/CIP-B102-Lab4/CIPB102_Lab4_Firstname_Lastname_USB.dd
Hash algorithm used	MD5 and SHA-256

Part C - Acquire the USB Image

Option B: Linux/Kali using dd

Step 1: identify the USB device, very carefully:

```
┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0   64M  0 loop
sda      8:0    0 80.1G  0 disk
└─sda1   8:1    0 80.1G  0 part /
```

lsblk

To compare, the new device that appears is as my USB, and Cross-check its size

```
┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ sudo fdisk -l
[sudo] password for kali:
Disk /dev/sda: 80.09 GiB, 86000000000 bytes, 167968750 sectors
Disk model: VMware Virtual S
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xfebdb644

Device     Boot Start       End   Sectors  Size Id Type
/dev/sda1  *     2048 167968749 167966702 80.1G 83 Linux


Disk /dev/loop0: 64 MiB, 67108864 bytes, 131072 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000
```

sudo fdisk -l

I used this as a second, independent confirmation of the device path and size before proceeding.

Output clearly showing which device is my USB, with size matching Part B.

Step 2: unmount it if it's auto-mounted:

```
┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ sudo umount /dev/loop0
umount: /dev/loop0: not mounted.
```

lsblk
sudo umount /dev/loop0
Out put shows not mounted

Step 3: To acquire the whole device (not just a partition)

```
┌──(kali㉿kali)-[~/CIP-B102-Lab4/virtual_usb]
└─$ sudo dd if=/dev/loop0 of=~/CIP-B102-Lab4/CIPB102_Lab4_yusuf_dada_USB.dd bs=4M status=progress conv=noerror,sync
67108864 bytes (67 MB, 64 MiB) copied, 14 s, 4.9 MB/s
16+0 records in
16+0 records out
67108864 bytes (67 MB, 64 MiB) copied, 14.276 s, 4.7 MB/s
```

```
sudo dd if=/dev/loop0 of=~/CIP-B102-Lab4/CIPB102_Lab4_yusuf_dada_USB.dd bs
=4M status=progress conv=noerror,sync
```

Part D - Hash Verification

Hash **both** the original USB and the acquired image, then compare.

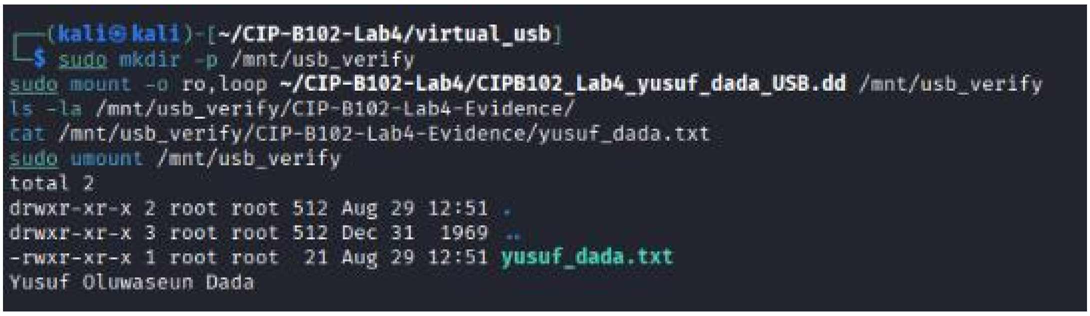
On Linux:


```
# Hash the source USB device directly
sudo md5sum /dev/loop0
sudo sha256sum /dev/loop0

# Hash the acquired image file
md5sum ~/CIP-B102-Lab4/CIPB102_Lab4_yusuf_dada_USB.dd
sha256sum ~/CIP-B102-Lab4/CIPB102_Lab4_yusuf_dada_USB.dd
```

Part E - Validate the Acquired Image

Prove your .dd file actually contains firstname_lastname.txt, this is the whole point of the lab: showing the acquisition preserved real, checkable evidence.

Option 2 - Linux read-only mounting (only if your USB was formatted with a standard file system like FAT32/exFAT that Linux can mount directly):



Output shows validated Acquired image, with checkable evidence

```
sudo mkdir -p /mnt/usb_verify
sudo mount -o ro,loop ~/CIP-B102-Lab4/CIPB102_Lab4_yusuf_dada_USB.dd /mnt/
usb_verify
ls -la /mnt/usb_verify/CIP-B102-Lab4-Evidence/
cat /mnt/usb_verify/CIP-B102-Lab4-Evidence/yusuf_dada.txt
sudo umount /mnt/usb_verify
```

The -o ro flag is important here, it mounts **read-only**, so am only viewing the image, never accidentally modifying the acquired evidence.